

COMPLETE DEVELOPER GUIDE

Laravel API

Best Practices

A production-ready reference covering authentication, routing, responses, security, performance, testing, versioning and deployment patterns.

01 Project Structure	02 Routing
03 Authentication	04 Request Validation
05 API Resources	06 Error Handling
07 Security	08 Performance
09 Testing	10 Versioning
11 Rate Limiting	12 Documentation

Laravel v7.0 – v12 · PHP 8.0+ · Compiled February 2026

CHAPTER 01

Project Structure & Organisation

Keep your API clean, scalable, and maintainable from day one.

Recommended Folder Structure

A well-organised project separates concerns clearly. Avoid putting business logic in controllers — use service classes, actions, or repositories to keep controllers thin and focused on HTTP.

Recommended app/ structure for a Laravel API

```
app/  
Http/  
Controllers/  
Api/  
V1/ PostController.php UserController.php  
V2/ PostController.php (new version endpoints)  
Requests/ # Form Request validation classes  
Resources/ # API Resource transformers  
Middleware/ # Custom middleware (ForceJson, ApiVersion...)  
Models/ # Eloquent models  
Services/ # Business logic (PostService, AuthService...)  
Actions/ # Single-responsibility action classes  
DTOs/ # Data Transfer Objects  
Enums/ # PHP 8.1 enums  
Exceptions/ # Custom exception classes  
Policies/ # Authorization policies  
routes/  
api.php # All API routes (install:api in v11+)  
tests/  
Feature/Api/ # Feature tests per endpoint  
Unit/ # Unit tests for services/helpers
```

Controller Best Practices

1

Architecture Keep Controllers Thin

Controllers should only handle HTTP: receive the request, call a service, and return a response. Move all business logic to Service or Action classes.

2

Structure Use Separate API Controller Namespace

Organise controllers under `App\Http\Controllers\Api\V1` to separate API logic from web controllers and support clean versioning.

3

Pattern One Action Per Controller (Optional)

For complex endpoints, consider single-action controllers using `__invoke()`. This improves readability and makes each endpoint's logic instantly clear.

Single-action controller example

```
// routes/api.php
Route::post('/posts/{post}/publish', PublishPostAction::class);
// App/Http/Controllers/Api/V1/PublishPostAction.php
class PublishPostAction extends Controller
{
    public function __invoke(Request $request, Post $post)
    {
        $this->authorize('publish', $post);
        $post->publish();
        return new PostResource($post);
    }
}
```

CHAPTER 02

Routing Best Practices

Design clean, predictable, RESTful API routes that are easy to maintain.

RESTful Route Design

Follow REST conventions consistently. Use nouns for resources, HTTP verbs for actions, and keep nesting to a maximum of two levels.

HTTP Method	URI	Controller Method	Description
GET	/api/v1/posts	index	List paginated posts
POST	/api/v1/posts	store	Create a new post
GET	/api/v1/posts/{post}	show	Get a single post
PUT	/api/v1/posts/{post}	update	Replace post (full)
PATCH	/api/v1/posts/{post}	update	Update post (partial)
DELETE	/api/v1/posts/{post}	destroy	Delete a post
GET	/api/v1/posts/{post}/comments	index	Nested resource

[routes/api.php](#) — Clean route organisation

```
// Group by version prefix
Route::prefix('v1')->name('v1.')->group(function () {
    // Public routes (no auth required)
    Route::get('posts', [PostController::class, 'index']);
    Route::get('posts/{post}', [PostController::class, 'show']);
    // Protected routes
    Route::middleware(['auth:sanctum', 'throttle:api'])->group(function () {
        Route::apiResource('posts', PostController::class)->except(['index', 'show']);
        Route::apiResource('comments', CommentController::class);
        // Scoped nested resources (v9+): ensures comment belongs to post
        Route::scopeBindings()->group(function () {
            Route::apiResource('posts.comments', PostCommentController::class);
        });
    });
});
```

Route Model Binding Tips

■ Use Implicit Binding

Let Laravel resolve models automatically from route parameters. Add `findOrFail` logic in the model's `resolveRouteBinding()` to customise.

● Best Practice

■ Custom Binding Key

Resolve by slug or UUID instead of ID. Add `getRouteKeyName()` to your model returning 'slug' or 'uuid'. Improves API security.

● Security

■ Scope Nested Bindings

Use `->scopeBindings()` on nested routes to ensure the child model belongs to the parent. Auto-returns 404 if ownership fails.

● v9+ Feature

■ Enum Bindings

PHP 8.1 backed enums work as route parameters. Invalid enum values automatically return 404 — no manual validation needed.

● v9+ Feature

CHAPTER 03

Authentication & Authorisation

Choose the right auth strategy and enforce permissions at every layer.

Laravel Sanctum — The Standard Choice

For most Laravel APIs — SPAs, mobile apps, and first-party consumers — Sanctum is the recommended authentication solution. Use Passport only when you need full OAuth2 for third-party integrations.

Complete Sanctum authentication flow

```
// 1. Install (Laravel 11+: php artisan install:api does this)
composer require laravel/sanctum
php artisan vendor:publish --provider='Laravel\Sanctum\SanctumServiceProvider'
php artisan migrate

// 2. Add HasApiTokens to User model
use Laravel\Sanctum\HasApiTokens;
class User extends Authenticatable {
    use HasApiTokens, HasFactory, Notifiable;
}

// 3. Login endpoint – issue a scoped token
Route::post('/login', function (Request $request) {
    $request->validate([
        'email' => 'required|email',
        'password' => 'required',
        'device' => 'required|string',
    ]);
    if (! Auth::attempt($request->only('email', 'password'))) {
        return response()->json(['message' => 'Invalid credentials'], 401);
    }
    $user = Auth::user();
    // Revoke any existing tokens for this device
    $user->tokens()->where('name', $request->device)->delete();
    $token = $user->createToken($request->device, ['posts:read', 'posts:write']);
    return response()->json(['token' => $token->plainTextToken]);
});

// 4. Logout – revoke current token only
Route::middleware('auth:sanctum')->delete('/logout', function (Request $request) {
    $request->user()->currentAccessToken()->delete();
    return response()->noContent();
});
```

Authorisation with Policies

Always use Laravel Policies for resource-level authorisation. Never perform ownership checks inline in controllers — extract them to policies for reusability and testability.

App\Policies\PostPolicy.php — Model policy

```
class PostPolicy
{
    // Any authenticated user can view posts
    public function viewAny(User $user): bool { return true; }
    // Only the author can update their post
    public function update(User $user, Post $post): bool
    {
        return $user->id === $post->user_id;
    }
    // Admins can delete; authors can delete their own
    public function delete(User $user, Post $post): bool
    {
        return $user->is_admin || $user->id === $post->user_id;
    }
}

// In controller – one line replaces all manual checks:
public function update(UpdatePostRequest $request, Post $post): PostResource
{
    $this->authorize('update', $post); // 403 if policy returns false
    $post->update($request->validated());
    return new PostResource($post->fresh());
}
```

Token Ability Checks: Always check token abilities (scopes) in addition to policies. A token may be valid but lack the required ability. Use `$request->user()->tokenCan('posts:write')` or the `CheckAbilities` middleware from Sanctum.

CHAPTER 04

Request Validation

Validate all incoming data with Form Requests — never trust client input.

Form Request Classes

Move all validation logic out of controllers and into dedicated Form Request classes. This keeps controllers clean and makes validation reusable and testable.

`App\Http\Requests\StorePostRequest.php`

```
class StorePostRequest extends FormRequest
{
    // Who is allowed to make this request?
    public function authorize(): bool
    {
        return $this->user()->tokenCan('posts:write');
    }

    // Validation rules
    public function rules(): array
    {
        return [
            'title' => ['required', 'string', 'min:3', 'max:255'],
            'body' => ['required', 'string', 'min:10'],
            'status' => ['required', Rule::enum(PostStatus::class)],
            'category_id' => ['required', 'integer', 'exists:categories,id'],
            'tags' => ['array', 'max:10'],
            'tags.*' => ['string', 'max:50'],
            'published_at' => ['nullable', 'date', 'after:now'],
        ];
    }

    // Custom error messages
    public function messages(): array
    {
        return [
            'category_id.exists' => 'The selected category does not exist.',
        ];
    }

    // Transform input before validation
    protected function prepareForValidation(): void
    {
        $this->merge(['title' => trim($this->title)]);
    }
}
```

Validation Best Practices

■ Always Use `Rule::enum()`

For PHP 8.1+ enum fields, use `Rule::enum(StatusEnum::class)` instead of 'in:' strings. It automatically validates against all enum values.

● PHP 8.1+

■■ Validate Nested Arrays

Use dot notation for nested inputs: 'items.*.product_id' => 'required|exists:products,id'. Don't skip validating array contents.

● Required

■ Use `prepareForValidation()`

Sanitise or transform input before rules run — trim whitespace, cast types, set defaults. Keeps rules clean.

● Best Practice

■ `validated()` Not `all()`

Always pass `$request->validated()` to models, never `$request->all()`. This prevents mass-assignment of unrequested fields.

● Security

CHAPTER 05

API Resources & Response Shaping

Transform Eloquent models into consistent, versioned JSON responses.

JsonResource Best Practices

Always use API Resources to transform responses. Never return raw Eloquent models or arrays — API Resources give you a stable contract with clients and make versioning painless.

App\Http\Resources\PostResource.php — Production example

```
class PostResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'type' => 'post', // JSON:API style
            'title' => $this->title,
            'slug' => $this->slug,
            'body' => $this->body,
            'status' => $this->status->value, // Enum value
            'reading_time' => $this->reading_time_minutes,
            // Conditional relationships – no N+1 if not loaded
            'author' => new UserResource($this->whenLoaded('user')),
            'tags' => TagResource::collection($this->whenLoaded('tags')),
            // Conditional fields based on permissions
            $this->mergeWhen($request->user()?->is_admin, [
                'ip_address' => $this->author_ip,
                'internal_notes' => $this->notes,
            ]),
            // Links (HATEOAS-lite)
            'links' => [
                'self' => route('v1.posts.show', $this->id),
            ],
            'created_at' => $this->created_at->toISOString(),
            'updated_at' => $this->updated_at->toISOString(),
        ];
    }
}
```

Response Status Code Standards

Status Code	When to Use	Example
200 OK	Successful GET, PUT, PATCH	Fetches or updates resource
201 Created	Successful POST (resource created)	POST /posts — new post created
204 No Content	Successful DELETE or no body needed	DELETE /posts/1
400 Bad Request	Malformed request, bad syntax	Missing required header
401 Unauthorized	No or invalid authentication token	Missing Bearer token
403 Forbidden	Authenticated but not authorised	Deleting another user post
404 Not Found	Resource does not exist	GET /posts/999
422 Unprocessable	Validation failed	Invalid email format
429 Too Many Req.	Rate limit exceeded	throttle middleware hit
500 Server Error	Unexpected application error	Uncaught exception

CHAPTER 06

Error Handling & Exceptions

Return consistent, informative JSON errors for every failure scenario.

Centralised Exception Handling

Configure all API exception handling in `bootstrap/app.php` (v11+) or `Handler.php`. Return a predictable JSON structure for all errors so API clients can handle them uniformly.

`bootstrap/app.php` — Centralised API error handling (v11+)

```
->withExceptions(function (Exceptions $exceptions) {
// Force JSON for all api/* routes
$exceptions->shouldRenderJsonWhen(
fn (Request $r) => $r->is('api/*') || $r->wantsJson()
);
// Custom response for domain exceptions
$exceptions->render(function (InsufficientCreditsException $e, Request $r) {
return response()->json([
'error' => 'insufficient_credits',
'message' => $e->getMessage(),
'credits' => $e->available,
], 402);
});
// Map model not found to clean 404
$exceptions->render(function (ModelNotFoundException $e) {
return response()->json([
'error' => 'not_found',
'message' => 'The requested resource was not found.',
], 404);
});
// Throttle exceptions
$exceptions->render(function (ThrottleRequestsException $e, Request $r) {
return response()->json([
'error' => 'rate_limit_exceeded',
'message' => 'Too many requests. Please slow down.',
'retry_after' => $e->getHeaders()['Retry-After'] ?? 60,
], 429)->withHeaders($e->getHeaders());
});
})
```

Consistent Error Response Schema

Every API error — validation, auth, not found, server error — should return the same predictable JSON envelope.

Standardised error response structure

```
// Validation error (422)
{
  "error": "validation_failed",
  "message": "The given data was invalid.",
  "errors": {
    "email": ["The email field is required."],
    "password": ["The password must be at least 8 characters."]
  }
}

// Auth error (401)
{
  "error": "unauthenticated",
  "message": "Please provide a valid API token."
}

// Domain error (402)
{
  "error": "insufficient_credits",
  "message": "You need 10 credits but only have 3.",
  "credits": 3
}
```

CHAPTER 07

Security Best Practices

Protect your API from common vulnerabilities and attack vectors.

Security Checklist

■ Always Use HTTPS

Never accept or serve API tokens over plain HTTP. Enforce HTTPS at the server/load-balancer level and redirect all HTTP traffic.

● Critical

■■ Force JSON Responses

Add ForceJson middleware to your API group. This prevents accidental HTML error pages being returned to API clients.

● Required

■ Rotate Tokens on Login

Delete existing tokens for the same device before issuing a new one. Prevents token accumulation and limits exposure from stolen tokens.

● Security

■ Mass Assignment Protection

Always define \$fillable or \$guarded on every model. Never use `Model::create($request->all())` — use `$request->validated()` only.

● Critical

■ Token Ability Scoping

Issue tokens with minimum required abilities. A read-only client should receive a token with only read abilities, not write access.

● Best Practice

■ Log Suspicious Activity

Log failed authentication attempts, repeated 403 errors, and unusual access patterns. Integrate with Laravel Telescope or an external SIEM.

● Monitoring

[App\Http\Middleware\ForceJsonResponse.php](#)

```
class ForceJsonResponse
{
    public function handle(Request $request, Closure $next): Response
    {
        // Tell Laravel this request expects JSON
        $request->headers->set('Accept', 'application/json');
        return $next($request);
    }
}

// Register in bootstrap/app.php (Laravel 11+)
->withMiddleware(function (Middleware $middleware) {
    $middleware->api(prepend: [ForceJsonResponse::class]);
})

// Or in app/Http/Kernel.php (Laravel 10 and below)
'api' => [
    \App\Http\Middleware\ForceJsonResponse::class,
    'throttle:api',
    \Illuminate\Routing\Middleware\SubstituteBindings::class,
],
```

CORS Configuration: Configure CORS in config/cors.php carefully. Never use 'allowed_origins' => ['*'] in production. Explicitly list your frontend domain(s). Set 'supports_credentials' => true only if using cookie-based Sanctum auth.

CHAPTER 08

Performance Optimisation

Build APIs that scale — avoid N+1, use caching, and paginate everything.

Eliminate N+1 Queries

The N+1 query problem is the most common API performance killer. Always eager-load relationships and use Laravel Debugbar or Telescope to catch stray queries in development.

PostController.php — Eager loading best practice

```
// BAD: Causes N+1 — 1 query for posts + 1 per post for user
public function index()
{
    return PostResource::collection(Post::paginate(15));
    // In PostResource: $this->user->name => QUERY PER POST
}

// GOOD: Eager-load everything the Resource will access
public function index()
{
    $posts = Post::with(['user', 'tags', 'category'])
        ->withCount('comments') // efficient count
        ->latest()
        ->paginate(15);
    return PostResource::collection($posts);
}

// ALSO GOOD: Prevent loading in Resource if not needed
// Use whenLoaded() in your Resource:
'author' => new UserResource($this->whenLoaded('user')),
// Only included if 'user' was eager-loaded — safe either way
```

Caching Strategy

Caching expensive API responses with tags

```
// Cache a resource list with tags for easy invalidation
public function index(Request $request)
{
    $cacheKey = 'posts:page:' . $request->get('page', 1);
    $posts = Cache::tags(['posts'])->remember($cacheKey, 300, function () {
        return Post::with(['user', 'tags'])->latest()->paginate(15);
    });
    return PostResource::collection($posts);
}

// When a post is created/updated/deleted – bust the cache
Cache::tags(['posts'])->flush();

// Cache individual resources
public function show(Post $post)
{
    $data = Cache::tags(['posts', "post:{$post->id}"])
        ->remember("post:{$post->id}", 600, fn () =>
        $post->load(['user', 'tags', 'comments'])
        );
    return new PostResource($data);
}
```

■ Always Paginate List Endpoints

Never return unbounded lists. Use `paginate(15)` for cursor-based or page-based pagination. Use `cursorPaginate()` for real-time feeds.

● Required

■ Use API Resource Caching

For expensive transformations, cache the serialised resource output directly. Set appropriate Cache-Control headers for client-side caching.

● Performance

■■ Select Only Needed Columns

Use `->select(['id','title','user_id'])` for list endpoints. Avoid `SELECT *` when only a few fields are needed in the response.

● Optimisation

■ Queue Heavy Operations

Never perform emails, PDF generation, or third-party API calls synchronously in a request. Dispatch jobs and return 202 Accepted.

● Pattern

CHAPTER 09

Testing API Endpoints

Write comprehensive feature tests — every endpoint, every edge case.

Feature Test Structure

Use Laravel's built-in HTTP testing to write expressive feature tests. Test all success paths, all validation failures, and all authorisation scenarios.

```
tests/Feature/Api/V1/PostControllerTest.php
```


Pro Tip: Use `Http::fake()` to mock all external HTTP calls in tests, `Queue::fake()` to assert jobs were dispatched, and `Mail::fake()` to verify emails were triggered — all without hitting real services.

CHAPTER 10

API Versioning

Version your API from day one — breaking changes are inevitable.

URI Versioning Strategy

URI versioning (`/api/v1/`, `/api/v2/`) is the most explicit and widely understood approach. Clients know exactly which version they are using, and server logs make version usage immediately visible.

routes/api.php — Clean URI versioning

```
// V1 — stable, maintained
Route::prefix('v1')->name('v1.')->group(function () {
    Route::middleware(['auth:sanctum', 'throttle:api'])->group(function () {
        Route::apiResource('posts', \App\Http\Controllers\Api\V1\PostController::class);
        Route::apiResource('users', \App\Http\Controllers\Api\V1\UserController::class);
        Route::apiResource('comments', \App\Http\Controllers\Api\V1\CommentController::class);
    });
});

// V2 — new features, evolved contracts
Route::prefix('v2')->name('v2.')->group(function () {
    Route::middleware(['auth:sanctum', 'throttle:api'])->group(function () {
        Route::apiResource('posts', \App\Http\Controllers\Api\V2\PostController::class);
        Route::apiResource('reactions', \App\Http\Controllers\Api\V2\ReactionController::class);
    });
});

// Tip: V2 controller can extend V1 and override only changed methods
// namespace App\Http\Controllers\Api\V2;
// class PostController extends \App\Http\Controllers\Api\V1\PostController
// { public function show(Post $post): PostResource { ... } }
```

Versioning Rules

■ Never Break Existing Versions

Once a version is public, only add to it — never remove fields, change types, or alter behaviour. Breaking changes go in a new version.

● Rule

■ Deprecation Notice Headers

When a version is deprecated, add a Deprecation header to responses: `Deprecation: true` and `Sunset: Sat, 01 Jan 2026 00:00:00 GMT`.

● Standard

■ Document Version Differences

Maintain a CHANGELOG per version. List what was added, changed, and removed in each version so clients can plan migration.

● Required

■■ Separate Resources per Version

Keep Resource classes separate per version (V1/PostResource, V2/PostResource). Avoid sharing resources between versions to prevent coupling.

● Architecture

CHAPTER 11

Rate Limiting

Protect your API from abuse with layered, dynamic rate limits.

Multi-Tier Rate Limiting

Implement rate limits at multiple levels: per-second for burst protection, per-minute for normal use, and per-plan for business tier enforcement.

App\Providers\AppServiceProvider.php — Rate limiters

```
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Support\Facades\RateLimiter;

public function boot(): void
{
    // General API limiter – applied to all API routes
    RateLimiter::for('api', function (Request $request) {
        $user = $request->user();
        return [
            Limit::perSecond(10)->by($user?->id ?: $request->ip()),
            Limit::perMinute($user?->rate_limit ?? 60)
                ->by($user?->id ?: $request->ip()),
        ];
    });

    // Strict limiter for auth endpoints (prevent brute-force)
    RateLimiter::for('auth', function (Request $request) {
        return Limit::perMinute(5)->by($request->ip())
            ->response(function () {
                return response()->json([
                    'error' => 'too_many_attempts',
                    'message' => 'Too many login attempts. Try again in 1 minute.',
                ], 429);
            });
    });

    // Upload limiter – expensive operations get stricter limits
    RateLimiter::for('uploads', function (Request $request) {
        return Limit::perHour(20)->by($request->user()?->id ?: $request->ip());
    });
}

// Apply in routes
Route::middleware(['auth:sanctum', 'throttle:api'])->group(fn() => ...);
Route::middleware('throttle:auth')->group(fn() => ...);
Route::middleware(['auth:sanctum', 'throttle:uploads'])->post('/upload', ...);
```

Rate Limit Headers: Laravel automatically adds X-RateLimit-Limit, X-RateLimit-Remaining, and Retry-After headers when throttled. Document these in your API so clients can implement backoff strategies.

CHAPTER 12

API Documentation & OpenAPI

Great APIs have great documentation — auto-generate it from your code.

Recommended Documentation Tools

Tool	Approach	Best For
Scribe (knuckleswtf/scribe)	Generates docs from controllers, Form Requests, and docblocks automatically.	Most Laravel projects — zero-config start
L5-Swagger (darkaonline)	OpenAPI 3.0 annotations in docblocks. Full Swagger UI integration.	Teams that need full OpenAPI spec
Scramble (dedoc/scramble)	Analyses PHP code statically — no annotations needed. Modern OpenAPI output.	Projects avoiding annotation clutter
Postman Collections	Manual or auto-generated via Scribe export. Great for team sharing.	Development and QA teams

Documentation Best Practices

■ Document All Responses

Document both success and error responses for every endpoint. Include status codes, response schema, and example payloads.

● Required

■ Authentication Examples

Include exact examples of how to obtain and use tokens. Show Bearer token format in Authorization header clearly.

● Required

■ Keep Docs in Sync

Use auto-generation tools so docs update with code. Add doc generation to your CI/CD pipeline to catch drift early.

● Best Practice

■ Versioned Documentation

Maintain separate docs per API version. Clearly mark deprecated endpoints and link to migration guides.

● Standard

CHAPTER REF

Quick Reference Cheatsheet

The most important rules at a glance — print and pin this page.

■ Structure

- ✓ Controllers only handle HTTP — move logic to Services/Actions
- ✓ Separate API controllers under `Api\V1\` namespace
- ✓ Use Form Request classes for all validation
- ✓ Use API Resources for all responses — never raw models

■ Security

- ✓ Enforce HTTPS at infrastructure level
- ✓ Add `ForceJsonResponse` middleware to API group
- ✓ Use `$request->validated()` never `$request->all()`
- ✓ Issue tokens with minimum required abilities
- ✓ Rotate tokens on each login — delete old device tokens
- ✓ Configure CORS with explicit allowed origins

■ Performance

- ✓ Always eager-load relationships with `with()`
- ✓ Always paginate list endpoints — never return unbounded results
- ✓ Cache expensive list queries with `Cache::tags()`
- ✓ Select only required columns for list views
- ✓ Queue all heavy operations — emails, PDFs, external APIs

■ Quality

- ✓ Test every endpoint: success path, validation, 401, 403, 404
- ✓ Use `RefreshDatabase` trait in feature tests
- ✓ Mock external HTTP calls with `Http::fake()`
- ✓ Assert exact JSON structure with `assertJsonStructure()`
- ✓ Run `php artisan test --profile` to find slow tests

■ API Design

- ✓ Use HTTP verbs correctly (GET/POST/PUT/PATCH/DELETE)
- ✓ Return 201 Created for POST, 204 No Content for DELETE
- ✓ Version from day one — `/api/v1/` prefix on all routes
- ✓ Use consistent error envelope: `{error, message, errors}`
- ✓ Add rate limiting to all endpoints — stricter on auth routes

